Struktur Dasar UI, Layouting, dan Komposisi Widget

Dosen	: IKetut Gunawan, S.KOM, M.T.I
Kode/Mata Kuliah	: KB1185/ Aplikasi Mobile
Bobot Mata Kuliah	: 3 Sks
Prodi	: Teknik Informatika
Pertemuan	: 4

Cakupan Materi

- Cara Flutter Menyusun UI
- Tiga aturan layout yang berlaku di seluruh Flutter
- Membangun UI Profil (Langkah demi Langkah)
- Refactoring UI: Memisahkan Komponen (Extract Widget)

Tujuan Pembelajaran

Rencana Aktivitas Kelas (150 menit)

Waktu	Aktivitas

1 Cara Flutter Menyusun UI

Sebagian besar error layout bukan disebabkan widget yang salah, melainkan karena salah memahami siapa yang menentukan ukuran.

1.1 Widget bukan kotak di layar

Di Flutter, widget hanyalah deskripsi konfigurasi. Widget tidak menyimpan posisi akhir atau ukuran piksel; ia hanya memberi tahu framework seperti apa tampilan yang diinginkan. Framework kemudian membuat **render object** yang benar-benar menghitung ukuran dan menggambar ke layar.

Karena itu, membaca kode Flutter berarti membaca pohon widget. Contoh sederhana berikut memasang satu judul dan satu baris berisi ikon dan teks.

Contoh 1.1 Pohon widget sederhana di dalam Scaffold

```
Scaffold(
  body: Column(
    children: [
      const Text('Katalog Kampus'),
      Row(
        children: const [
          Icon(Icons.school),
          Text('Fakultas Ilmu Komputer'),
        ],
      ),
    ],
  ),
)
```

Jika ditulis sebagai pohon, strukturnya seperti ini. Indentasi menunjukkan siapa induk dan siapa anak, bukan menunjukkan posisi di layar.

Contoh 1.2 Bentuk pohon dari kode sebelumnya

```
Scaffold
+- Column
   +- Text "Katalog Kampus"
   +- Row
      +- Icon(Icons.school)
      +- Text "Fakultas Ilmu Komputer"
```

1.2 Tiga aturan layout yang berlaku di seluruh Flutter

Aturan layout di Flutter dapat dirangkum dalam satu kalimat: “Constraints go down. Sizes go up. Parent sets position.” Constraints turun: Induk memberikan batas minimum dan maksimum ukuran kepada anak.

Flutter memakai satu arah perhitungan yang konsisten. Jika tiga aturan ini dipahami, sebagian besar pesan error menjadi mudah ditebak.

Tabel 1.1 Tiga aturan layout Flutter

Aturan	Arti	Akibat di praktik
Constraints turun	Induk memberi batas minimum dan maksimum kepada anak.	Anak tidak mungkin lebih besar dari batas yang diberikan induk.
Ukuran naik	Anak memilih ukuran di dalam batas itu, lalu melaporkannya ke induk.	Induk mengetahui berapa ruang yang sudah terpakai.
Induk yang menentukan posisi	Setelah ukuran diketahui, induk	Anak tidak bisa memindahkan dirinya sendiri; karena itu

	menempatkan anak pada posisinya.	ada `Align`, `Center`, dan `Stack`.

KENAPA CENTER DAN ALIGN ADA

Karena anak tidak menentukan posisinya sendiri, dibutuhkan widget yang secara eksplisit mengatur posisi anak. `Center` dan `Align` adalah induk yang tugasnya hanya menempatkan satu anak di dalam ruangnya.

Code :

```
void main() {
  runApp(const MaterialApp(
    home: LayoutRulesExample(),
  ));
}
```

```
class LayoutRulesExample extends StatelessWidget {
  const LayoutRulesExample({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Tiga Aturan Layout Flutter'),
      ),
      body: MyPages(),
   );
 }
}
```

```
class MyPages1 extends StatelessWidget {
  const MyPages1({super.key});

  @override
  Widget build(BuildContext context) {
    return SingleChildScrollView(
        child: Padding(
          padding: const EdgeInsets.all(16.0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              const Text(
                '1 & 2. Aturan Constraints (Batas) dan Size (Ukuran)',
                style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
              ),
              const SizedBox(height: 10),
              const Text('Induk memaksa anak berada di batas 100x100 hingga
200x200. Anak meminta ukuran 500x50, tapi dipaksa mengikuti batas induk (mentok
di lebar 200, tinggi 100).'),
              const SizedBox(height: 10),

              // CONTOH ATURAN 1 & 2
              ConstrainedBox(
                // INDUK memberikan constraints ke bawah (Constraints turun)
                constraints: const BoxConstraints(
                  minWidth: 100,
                  maxWidth: 200,
                  minHeight: 100,
                  maxHeight: 200,
                ),
                // ANAK mencoba menentukan ukuran (Ukuran naik)
                child: Container(
                  width: 500, // Mencoba lebih besar dari maxWidth (Gagal,
dipaksa 200)
                  height: 50, // Mencoba lebih kecil dari minHeight (Gagal,
dipaksa 100)
                  color: Colors.blue,
                  child: const Text(
                    'Saya minta 500x50, tapi dapatnya 200x100',
                    style: TextStyle(color: Colors.white),
                  ),
                ),
              ),
            ],
          ),
        ),
      );
  }
}
```

```
class MyPages2 extends StatelessWidget {
  const MyPages2({super.key});

  @override
  Widget build(BuildContext context) {
    return SingleChildScrollView(
        child: Padding(
          padding: const EdgeInsets.all(16.0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
             // const Divider(height: 40, thickness: 2),

              const Text(
                '3. Aturan Posisi (dengan Center)',
                style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
              ),
              const SizedBox(height: 10),
              const Text('Induk abu-abu berukuran 200x200. Kotak merah 80x80 tidak bisa menaruh dirinya di
tengah, jadi kita gunakan widget Center.'),
              const SizedBox(height: 10),

              // CONTOH ATURAN 3 (CENTER)
              Container(
                width: 200,
                height: 200,
                color: Colors.grey[300], // Induk yang memiliki ruang
                // Induk yang menentukan posisi
                child: Center(
                  child: Container(
                    width: 80,
                    height: 80,
                    color: Colors.red,
                    child: const Text('Center', style: TextStyle(color: Colors.white)),
                  ),
                ),
              ),

              const Divider(height: 40, thickness: 2),

              const Text(
                '3. Aturan Posisi (dengan Align)',
                style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
              ),
              const SizedBox(height: 10),
              const Text('Induk abu-abu berukuran 200x200. Kotak hijau 80x80 ditempatkan di pojok kanan
bawah menggunakan widget Align.'),
              const SizedBox(height: 10),

              // CONTOH ATURAN 3 (ALIGN)
              Container(
                width: 200,
                height: 200,
                color: Colors.grey[300], // Induk yang memiliki ruang
                // Induk yang menentukan posisi
                child: Align(
                  alignment: Alignment.bottomRight,
                  child: Container(
                    width: 80,
                    height: 80,
                    color: Colors.green,
                    child: const Text('Align', style: TextStyle(color: Colors.white)),
                  ),
                ),
              ),
            ],
          ),
        ),
      );
  }
}
```

```
class MyPages3 extends StatelessWidget {
  const MyPages3({super.key});

  @override
  Widget build(BuildContext context) {
    return SingleChildScrollView(
        child: Padding(
          padding: const EdgeInsets.all(16.0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              const Text(
                '3. Aturan Posisi (dengan Align)',
                style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
              ),
              const SizedBox(height: 10),
              const Text('Induk abu-abu berukuran 200x200. Kotak hijau 80x80
ditempatkan di pojok kanan bawah menggunakan widget Align.'),
              const SizedBox(height: 10),

              // CONTOH ATURAN 3 (ALIGN)
              Container(
                width: 200,
                height: 200,
                color: Colors.grey[300], // Induk yang memiliki ruang
                // Induk yang menentukan posisi
                child: Align(
                  alignment: Alignment.bottomRight,
                  child: Container(
                    width: 80,
                    height: 80,
                    color: Colors.green,
                    child: const Text('Align', style: TextStyle(color:
Colors.white)),
                  ),
                ),
              ),
            ],
          ),
        ),
      );
  }
}
```

Penjelasan Kode terhadap Aturan:

1. **Constraints Turun & Ukuran Naik (`ConstrainedBox` vs `Container`)**
   - Di contoh pertama, `ConstrainedBox` (Induk) berkata: *"Kamu tidak boleh lebih sempit dari 100x100, dan tidak boleh lebih lebar dari 200x200"*.
   - `Container` biru (Anak) berkata: *"Saya ingin ukuran 500x50"*.
   - Karena **Constraints Turun**, anak terpaksa tunduk. Anak kemudian melaporkan ukurannya (**Ukuran Naik**) sebesar 200 (karena dipotong dari 500 ke batas maksimal) x 100 (karena didorong dari 50 ke batas minimal).
2. **Induk Menentukan Posisi (Peran `Center`)**
   - Di contoh kedua, ada `Container` abu-abu (200x200). `Container` merah (80x80) tidak punya cara atau parameter untuk berkata *"tolong taruh saya di tengah"*.
   - Oleh karena itu, kita membungkusnya dengan `Center`. `Center` mengambil seluruh ruang 200x200 dari induknya, melihat anaknya berukuran 80x80, menghitung sisa ruang, dan memposisikan anak tersebut pas di tengah.
3. **Induk Menentukan Posisi (Peran `Align`)**
   - Sama seperti `Center`, kotak hijau (80x80) tidak bisa mengatur dirinya sendiri.
   - `Align` memberitahu Flutter: *"Ambil seluruh ruang yang diberikan induk abu-abu, lalu taruh anak saya di posisi `bottomRight` (Kanan Bawah)"*.

-

2 Membangun UI Profil (Langkah demi Langkah)

Pendekatan ini memperlakukan setiap elemen di layar sebagai *widget*. Secara arsitektur, praktik terbaik untuk membangun halaman adalah memisahkan tanggung jawab secara tegas (*Feature-First*): file utama hanya bertugas mengatur konfigurasi global, sedangkan logika visual dan tata letak diisolasi di dalam file halaman spesifik.

Berikut adalah alur penyusunan halaman profil dari nol:

1.Inisialisasi Root Aplikasi:main.dart.

Setiap aplikasi Dart dimulai dari fungsi void main(). Fungsi ini mengeksekusi runApp(), yang mengikat *root widget* (MaterialApp) ke layar perangkat. MaterialApp menangani tema dasar, navigasi, dan menunjuk ProfilePage sebagai halaman pertama yang dimuat.

2.Membangun Kerangka Halaman:profile_page.dart.

Untuk membangun sebuah halaman penuh, kita menggunakan Scaffold. Widget ini menyediakan struktur standar Material Design, memberikan ruang khusus untuk bilah atas (appBar) dan area konten utama (body).

3.Menyusun Tata Letak Vertikal: Penggunaan Column.

Profil umumnya menampilkan foto, nama, dan bio dari atas ke bawah. Kita membungkus elemen-elemen tersebut menggunakan Column di dalam area body, lalu menggunakan SizedBox untuk memberikan spasi vertikal antar elemen.

4.Menerapkan Komponen Visual: CircleAvatar & Text.

Foto profil dirender menggunakan CircleAvatar yang secara otomatis memotong gambar menjadi lingkaran. Informasi teks ditampilkan menggunakan widget Text dengan properti TextStyle untuk mengatur ukuran, ketebalan, dan hierarki visual (nama lebih besar dari deskripsi).

Berikut adalah implementasi kode lengkap yang dipisahkan menjadi dua file agar terstruktur dan mudah dikembangkan:

```
void main() {
  // Titik eksekusi pertama
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Aplikasi Profil',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.blueAccent),
      ),
      // Menunjuk ProfilePage sebagai halaman utama
      home: const ProfilePage(),
    );
  }
}
```

```
class ProfilePage extends StatelessWidget {
  const ProfilePage({super.key});

  @override
  Widget build(BuildContext context) {
    // Scaffold memberikan struktur dasar halaman
    return Scaffold(
      appBar: AppBar(
        title: const Text('Profil Pengguna'),
        centerTitle: true,
        backgroundColor: Colors.blueAccent,
        foregroundColor: Colors.white,
      ),
      // Center memastikan keseluruhan profil berada di tengah layar
      body: Center(
        // Column menyusun anak-anaknya dari atas ke bawah
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const CircleAvatar(
              radius: 60,
              backgroundColor: Colors.grey,
              child: Icon(Icons.person, size: 60, color: Colors.white),
            ),
            const SizedBox(height: 20),
            const Text(
              'I Ketut Gunawan, S.Kom, M.T.I.',
              style: TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(height: 8),
            Text(
              'Senior Backend Engineer & Dosen',
              style: TextStyle(
                fontSize: 16,
                color: Colors.grey[700],
              ),
            ),
            const SizedBox(height: 32),
            // Tombol interaksi standar
            ElevatedButton.icon(
              onPressed: () {},
              icon: const Icon(Icons.edit),
              label: const Text('Edit Profil'),
            )
          ],
        ),
      ),
    );
  }
}
```

4 Refactoring UI: Memisahkan Komponen (Extract Widget)

Memecah UI yang panjang menjadi Custom Widget (Extract Widget) adalah praktik standar di Flutter. Pendekatan ini membuat kode lebih modular, mudah dibaca, dan komponennya dapat digunakan ulang (reusable) di halaman lain.

Berikut adalah langkah-langkah memecah UI profil menjadi komponen terpisah:

1.Identifikasi Komponen:

Tentukan bagian UI mana yang memiliki satu tanggung jawab spesifik. Pada halaman profil kita, ada dua bagian utama yang bisa dipisah: **Area Foto** (CircleAvatar) dan **Area Informasi** (Nama & Jabatan).

2.Buat Kelas StatelessWidget Baru:

Buat kelas baru yang merupakan turunan dari StatelessWidget untuk masing-masing bagian tersebut (misalnya ProfileAvatar dan ProfileInfo).

3.Gunakan Parameter (Passing Data):

Agar komponen lebih fleksibel dan tidak *hardcoded*, tambahkan parameter (variabel) di *constructor* kelas baru tersebut. Ini memungkinkan kita mengirim nama atau foto yang berbeda-beda nantinya.

4.Panggil di Halaman Utama:

Gantikan blok kode yang panjang di dalam ProfilePage dengan memanggil Custom Widget yang baru saja dibuat.

5 Tugas Memisahkan Komponen (Extract Widget)

Rencana Tugas Praktikum: Membangun UI Aplikasi Profil dengan Flutter (4 Hari) dengan commit management

Instruksi Utama Praktikum: Dalam tugas ini, Anda tidak hanya sekadar membuat antarmuka aplikasi, tetapi juga melatih kemampuan analisis dan *troubleshooting* kode. Anda diwajibkan untuk:

1. Membaca dan Memahami: Pahami alur dan logika dari setiap contoh kode yang diberikan.
2. Menyalin Bertahap (Step-by-Step): Tulis ulang kode secara berurutan sesuai target harian. Biasakan mengetik mandiri agar terbiasa dengan sintaks dan struktur *widget* Flutter.
3. Memperbaiki Error (Debugging): Perhatian, terdapat beberapa *error/bug* yang sengaja disisipkan pada contoh kode panduan. Tugas Anda adalah menemukan, menganalisis, dan memperbaiki kode tersebut agar aplikasi dapat di-kompilasi dan berjalan sempurna.

Rencana Tugas Praktikum : 4 Hari (Commit Management)

Tugas praktik pembuatan UI profil ini dibagi ke dalam 4 hari dengan alur commit berstandar industri (Conventional Commits). Pendekatan yang digunakan murni statis (StatelessWidget) agar mahasiswa fokus pada komposisi dan tata letak.

Hari 1: Kerangka Dasar & Refactoring Awal

· Pagi: Membuat Kerangka Utama
Tugas: Membuat main.dart dan profile_page.dart dengan Scaffold, AppBar, CircleAvatar, dan Text.
Commit: feat: create basic profile page structure with avatar and text

· Siang: Extract Widget (Bagian 1)
Tugas: Memecah Column dengan membuat Custom Widget ProfileAvatar dan ProfileInfo.
Commit: refactor: extract avatar and info into custom widgets

Hari 2: Menambahkan Komponen Statistik Profil

· Pagi: Membangun Baris Statistik
Tugas: Menambahkan widget Row yang berisi 3 buah Column angka dan label (Proyek, Followers, Following).
Commit: feat: add profile statistics row

· Siang: Extract Widget (Bagian 2)
Tugas: Mengekstrak bagian Row menjadi widget mandiri ProfileStats dan StatItem.
Commit: refactor: extract statistics UI into ProfileStats and StatItem widgets

Hari 3: Menambahkan Daftar Pengalaman (ListView)

· Pagi: Implementasi ListView & Expanded
Tugas: Menambahkan Expanded yang membungkus ListView.builder untuk menampilkan dummy list pengalaman kerja.
Commit: feat: add scrollable experience list using ListView.builder

· Siang: Merapikan Tampilan Item List (Card)
Tugas: Mengekstrak baris dalam ListView ke Custom Widget ExperienceCard dengan ikon dan Card elevation.
Commit: refactor: extract list tile into ExperienceCard widget and apply styling

Hari 4: Finishing & Interaksi Statis (SnackBar)

· Pagi: Penyesuaian Tata Letak
Tugas: Merapikan padding dan jarak (SizedBox) antar komponen agar lebih proporsional.
Commit: style: adjust padding and spacing for profile components

· Siang: Penambahan Aksi Statis pada Tombol
Tugas: Menambahkan fungsi statis pada tombol Edit Profil untuk memunculkan SnackBar (pesan bahwa fitur belum aktif).
Commit: feat: add static edit button with snackbar notification

Contoh Commit message

Hari 1: Kerangka Dasar & Refactoring Awal

Fokus: Membangun UI monolitik pertama kali, lalu langsung mempraktikkan materi Extract Widget agar kode rapi.

- Pagi: Membuat Kerangka Utama
  - Tugas: Membuat main.dart dan profile_page.dart. Menyusun Scaffold, AppBar, dan menyusun CircleAvatar serta Text di dalam Column tunggal (belum dipisah).
  - Pesan Commit: feat: create basic profile page structure with avatar and text
- Siang/Malam: Extract Widget (Bagian 1)
  - Tugas: Memecah Column yang panjang dengan membuat Custom Widget ProfileAvatar dan ProfileInfo (menerapkan parameter passing data untuk nama dan role).
  - Pesan Commit: refactor: extract avatar and info into custom widgets

Hari 2: Menambahkan Komponen Statistik Profil

Fokus: Melatih penggunaan Row, Expanded, dan mempraktikkan kembali Extract Widget untuk komponen baru.

- Pagi: Membangun Baris Statistik

  - Tugas: Menambahkan widget Row di bawah tombol Edit Profil. Row ini berisi 3 buah Column yang masing-masing menampilkan angka (misal: "12") dan label (misal: "Proyek", "Followers", "Following").
  - Pesan Commit: feat: add profile statistics row
- Siang/Malam: Extract Widget (Bagian 2)
  - Tugas: Mengekstrak bagian Row tersebut menjadi widget mandiri bernama ProfileStats, dan mengekstrak masing-masing item angka/label menjadi widget StatItem agar reusable.
  - Pesan Commit: refactor: extract statistics UI into ProfileStats and StatItem widgets

Hari 3: Menambahkan Daftar Pengalaman (ListView)

Fokus: Menggabungkan tata letak statis di atas dengan daftar gulir dinamis di bawahnya tanpa memicu error overflow.

- Pagi: Implementasi ListView & Expanded
  - Tugas: Menambahkan widget Expanded di bagian bawah layar yang membungkus ListView.builder. Mengisi ListView dengan list array dummy data (misal: daftar pengalaman backend engineering).
  - Pesan Commit: feat: add scrollable experience list using ListView.builder
- Siang/Malam: Merapikan Tampilan Item List (Card)
  - Tugas: Mengekstrak desain baris dalam ListView ke dalam Custom Widget bernama ExperienceCard. Menambahkan ikon, padding, dan efek bayangan (elevation) pada kartu tersebut.
  - Pesan Commit: refactor: extract list tile into ExperienceCard widget and apply styling

Hari 4: State Management Dasar & Interaktivitas

Fokus: Mengubah UI dari yang semula statis (hanya tampilan) menjadi dinamis dan bisa menerima input.

- Pagi: Konversi ke StatefulWidget
  - Tugas: Mengubah ProfilePage yang semula StatelessWidget menjadi StatefulWidget. Mendeklarasikan variabel userName di dalam state class agar nilainya tidak lagi di-hardcode.
  - Pesan Commit: refactor: convert ProfilePage to StatefulWidget for dynamic data
- Siang/Malam: Implementasi Form Dialog Edit
  - Tugas: Memberikan aksi pada onPressed di tombol "Edit Profil" untuk memunculkan showDialog berisi TextFormField. Saat tombol "Simpan" ditekan di dialog, gunakan setState() untuk memperbarui variabel userName di layar utama.
  - Pesan Commit: feat: implement edit profile dialog with state update